

XPU量化推理简介

目录

- 1.XPU简介
- 2.一般计算硬件上的量化方案
- 3.XPU上现在的Gemm/Conv2d方案
- 4.存在的挑战

 本页面由边俊柯于2022/06/17迁移自wiki 缪天翔空间

1.XPU简介

XPU是一个深度学习计算加速器，主要包含Cluster和SD-CDNN两大计算部件。Cluster包含完备的加减乘除数学计算指令，类似CUDA中的Stream Multiprocessor，可以用C++语法进行编程，灵活完成一般Kernel计算。SD-CDNN是专用计算单元，可以高效处理Gemm和Conv2d这些计算密集型算子，开发者在Kernel中插入intrinsic函数调用其功能。

Cluster处理的主要数据类型是fp32。而SD-CDNN主要接受类型是fp32的输入数据（权重预先转成int8/int16），内部转成int8/int16计算，最后再输出fp32。因此，量化推理主要集中在SD-CDNN，也就是Gemm和Conv2d这两个Op上。

昆仑芯片核心部件：Cluster+SDNN

Cluster：

- 类GPU 架构
- 1Tops 的 FP32算力
- 承担通用计算任务

SDNN：

- 软件定义的 NN 引擎,提供灵活的编程性
- 256T/64Tops 的 Int8/16算力
- 支持卷积、矩阵运算等核心算子



使用Cluster加速计算

data



4 clusters * 16 cores

```
int main() {
    /// ...
    auto ctx = api::create_context();
    float* data_xpu = nullptr;
    xpu_malloc(&data_xpu, len * sizeof(float));

    xpu_memcpy(data_xpu, data_cpu, len *
        sizeof(float), XPU_HOST_TO_DEVICE);

    relu<<<4, 16>>>(ctx, data_xpu, len);

    xpu_memcpy(data_cpu, data_xpu,
        len * sizeof(float), XPU_DEVICE_TO_HOST);
    xpu_free(data_xpu);
}
```

在设备上分配空间

输入拷贝到设备端

获取输出
释放设备空间

在设备上运行relu,
启动4个 cluster,
每个 cluster 16个
核

relu.xpu



Cluster编程模型

data



显式搬运数据到
Local Memory 上

```
/// relu.xpu
__global__ void relu_xpu(float* data, int n) {
    int tid = core_id() * cluster_num() + cluster_id();
    int nthreads = cluster_num() * core_num();
    const int bufsize = 1024;
    __local__ float buffer[bufsize];
    for (int i = tid * bufsize; i < n; i += nthreads * bufsize) {
        GM2LM(data + i, buffer, bufsize * sizeof(float));
        for (int j = 0; j < bufsize; j++) {
            buffer[j] = buffer[j] > 0 ? buffer[j] : 0;
        }
        LM2GM(buffer, data + i, bufsize * sizeof(float));
    }
}
```

根据tid进行
数据划分

计算

显式写回到
Global Memory



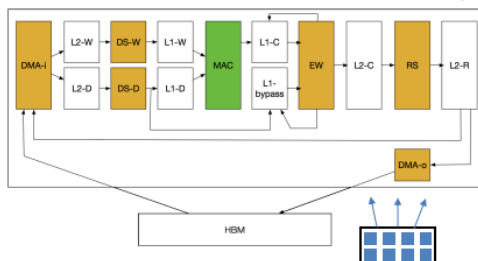
SDNN硬件架构

专用硬件单元

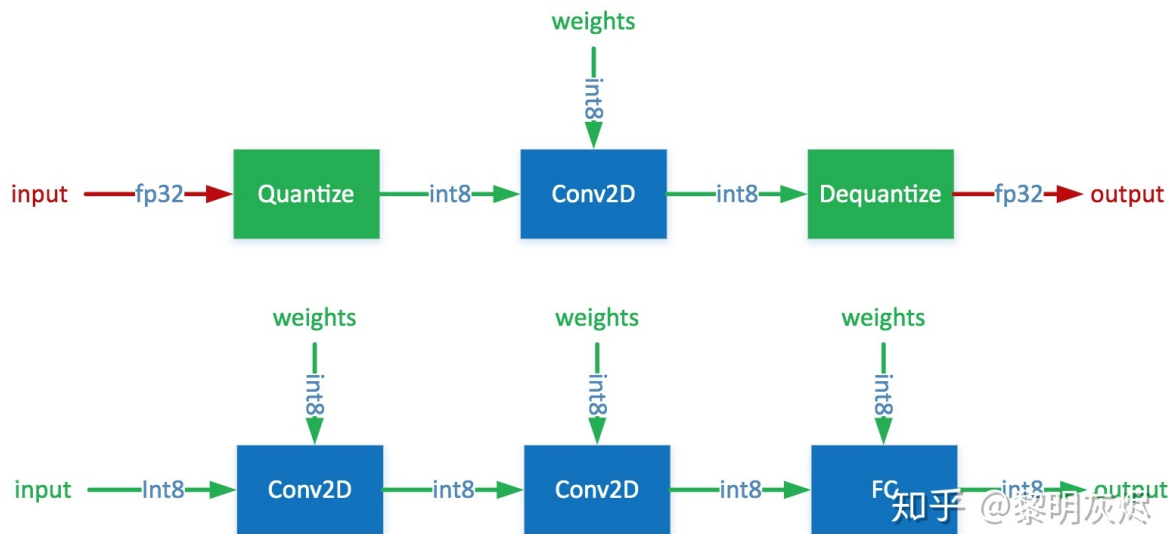
- 卷积窗口提取、矩阵转置单元(DS)
- 矩阵乘法单元(MAC)
- 向量计算单元(EW)
- 大容量片上 SRAM

通用核

- 执行用户代码，类 C/C++ 语言
- 不做实际计算，只做控制(循环、分支、地址计算等)
- 将具体计算任务发到相应的硬件单元去运行



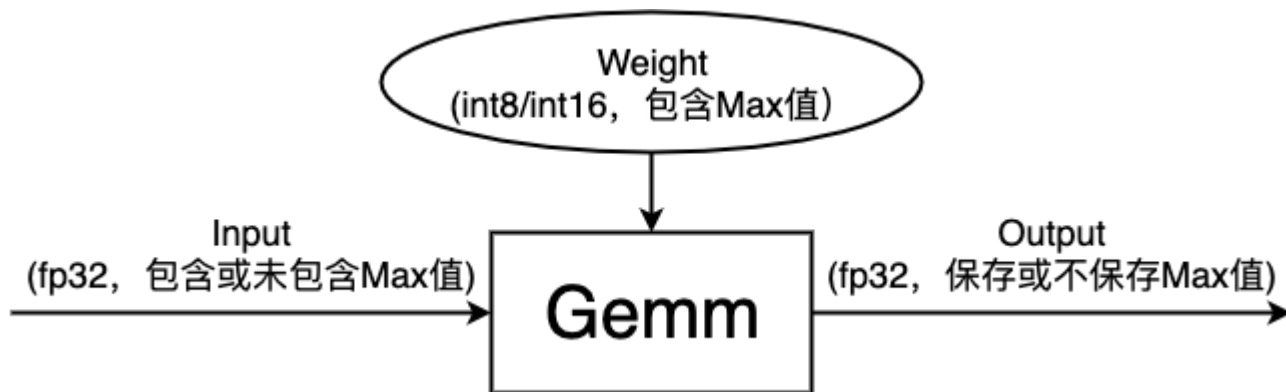
2.一般计算硬件上的量化方案



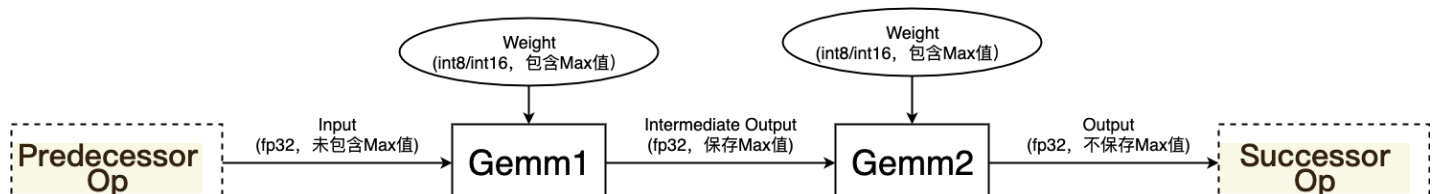
上半部分属于部分Op量化方案，推理引擎分析计算graph，在需要量化的Op前后分别插入Quantize和Dequantize Op，Op权重也预先量化好，这样量化Op便可以前后输入输出为fp32的Op无缝衔接。Quantize和Dequantize需要 $scale$ 和 $zero\ point$ 这两个标量参数，这个一般来源于经验数据或跑一些Sample数据后的统计值（这个过程也被称为Calibration校准）。

下半部分属于全网量化，暂不考虑。

3.XPU上现在的Gemm/Conv2d方案



1. 输入数据为fp32，如未传入Max值，Kernel内部先隐式Findmax（开销较大），将输入数据定点化转为int8/int16；
2. 权重预先处理好，传入int8/int16和Max值；
3. Gemm计算得到int48；
4. 根据输入数据和权重的Max值将int48反向转回fp32，如果选择保存Max值则对fp32再Findmax；



这是多个Op连在一起的情形，现在手写的大Fuse Kernel中都要保存中间计算结果的Max值，可以显著提高Kernel执行效率。

4.存在的挑战

- 需要为推理引擎增加Target专用Pass，把原始Graph转为对XPU友好的Graph（输入数据是fp32，权重是int8/int16，一般Graph中的Op不会这样）；
- 需要在推理引擎层面处理好前序Op的Max值，如果要提高多个连续Gemm Op性能的话（现在一般通过手写的大Fuse Kernel来提升ResNet和Bert模型性能）；